
Supervised and Self-Supervised Learning on Food-251

Anonymous Author(s)

Affiliation, Address

anon.email@example.org

Abstract

1 We train a residual convolutional network from scratch to classify the 251 fine-
2 grained food categories of FoodX-251, under a hard budget of ten million trainable
3 parameters and a single 8GB laptop GPU. We search among parameter-matched
4 trunk variants using short, fixed-seed proxy runs on a held-out development split,
5 then ablate six training-recipe axes – learning rate, augmentation strength, Mixup/
6 CutMix, an exponential moving average, a noise-robust loss, batch size, and the
7 random-resized-crop scale floor – against a plain label-smoothed cross-entropy
8 baseline, none of which improve on it. The resulting recipe, trained for a full 90
9 epochs, reaches 63.83% top-1 / 87.39% top-5 accuracy on a held-out test split
10 touched exactly once. We outline a planned comparison against self-supervised
11 pretraining under an equal-GPU-hour budget, reported where results exist and
12 marked otherwise. Throughout, we find that measured wall-clock and memory
13 behavior on commodity hardware – channel-width alignment, spatial resolution,
14 and thermal throttling – shape the final design as much as raw parameter or FLOP
15 counts do.

16 1 Introduction

17 FoodX-251 is a fine-grained food image classification benchmark spanning 251 categories, built from
18 a large, web-crawled, noisily labeled training set with a smaller, manually curated and clean valida-
19 tion set [1]. Fine-grained food categories are hard to separate because inter-class visual differences
20 (plating, preparation, garnish) can be subtler than the intra-class variation induced by lighting, angle,
21 and portioning, and the label noise inherent to a web-crawled training set compounds the problem.

22 This work targets FoodX-251 under two constraints imposed by the setting rather than chosen for
23 effect: the classifier is trained from scratch, with no ImageNet or other pretraining, under a budget
24 of fewer than 10,000,000 trainable parameters, and every run happens on a single consumer laptop
25 GPU with 8GB of VRAM. Neither constraint is negotiable, and together they shape everything
26 that follows: an architecture search that screens on measured throughput and memory before ever
27 measuring accuracy (Section 5.1), and a training recipe built to spend the wall-clock the 8GB budget
28 buys, not just the parameter budget itself.

29 We make four contributions, in the order the underlying work was carried out: a measured, budget-
30 constrained architecture search over residual-trunk variants (Section 5.1); a from-scratch training
31 recipe modernized with mixed-precision training, a cosine schedule, and label smoothing, plus a
32 set of opt-in extensions – stronger augmentation, Mixup/CutMix, an exponential moving average, a
33 noise-robust loss – implemented and ablated, none of which improve on the plain recipe (Section 5.2);
34 a full-length training run validating the selected trunk and tuned recipe (Section 5.3); and a planned
35 comparison of supervised training against self-supervised pretraining under an equal-GPU-hour
36 budget (Section 5.4).

37 One design choice is worth flagging before the rest. Despite FoodX-251’s long-tailed reputation, the
38 training split we measured is close to class-balanced (median 471 images per class; a single 34-

image class drives the oft-cited 19.3x imbalance ratio), so class-balanced losses, logit adjustment, and long-tail-specific training are out of scope here – the real distributional problem is label noise, not long-tailedness (Section 3). More generally: the architecture search, recipe ablation, and full training run are complete at the time of writing; the self-supervised comparison is in progress, and is marked accordingly below rather than reported with numbers we have not measured.

2 Related work

We organize related work into three lanes, matching the three axes of this project: architecture, training recipe, and pretraining objective.

Parameter-efficient architecture design. Recent efficient-CNN work targets mobile and edge deployment under similar per-parameter and per-FLOP pressure: MobileNetV4 [2] and MobileOne [3] optimize explicitly for on-device latency rather than parameter count alone, and ConvNeXt V2 [4] revisits pure-convolutional design at a scale where masked-autoencoder pretraining is available – a regime this project’s from-scratch constraint excludes. None of these target fine-grained food classification, and our own literature search turned up no source addressing FoodX-251, Food-101, or Food-2K under a hard parameter budget (research/2026-07-30-architecture-brief.md) – the architecture decisions in Section 4 rest on general efficient-CNN principles and our own measurements rather than a food-specific prior.

Augmentation and label-noise robustness. RandAugment [5] and TrivialAugment [6] establish that automated or tuning-free augmentation policies match or beat searched policies at near-zero search cost; mixup [7] and CutMix [8] are complementary regularizers that mix inputs and labels rather than choosing among fixed operators. Generalized Cross Entropy [9] gives a noise-robust alternative to label-smoothed cross-entropy, directly relevant to FoodX-251’s web-crawled training labels. As with the architecture literature, none of these ablate sub-10M-parameter models trained for tens of epochs (research/2026-07-30-augmentation-noise-brief.md) – whether the accepted wisdom transfers to that regime is precisely what Section 5.2 is designed to check.

Self-supervised pretraining. SimCLR [10] established contrastive pretraining as a strong alternative to supervised training but relies on a large negative-sample batch to work well; at this project’s 8GB budget, reachable batch sizes fall below where SimCLR is known to degrade, which is why it is excluded here. BYOL [11], SimSiam [12], and Barlow Twins [13] instead avoid negative pairs entirely and remain viable at small batch sizes, making them the basis for the pretraining design in Section 5.4.

3 Dataset and protocol

FoodX-251 [1] contains 251 fine-grained food categories: a large, web-crawled training set with noisy labels, and a smaller, manually curated validation set of 11,994 images.

Class balance. Despite FoodX-251’s long-tailed reputation, per-class training counts are close to balanced: median 471 images per class, 10th/90th percentile 366/580, and exactly one class below 200 images (34) – that single outlier drives the frequently cited 19.3x imbalance ratio. We therefore do not use class-balanced losses, logit adjustment, or decoupled classifier retraining; the real distributional challenge is label noise, since training labels are web-crawled and validation labels are clean, not long-tailedness.

Validation split. Ablations and checkpoint selection must never touch the number this report ultimately quotes, or that number is implicitly selected-on. We split the 11,994-image validation set 50/50, stratified per class (every one of the 251 classes lands within one image of an exact half), into val-dev (6,063 images) and val-test (5,931 images), deterministically: src/make_val_split.py is a pure function of the official validation labels and a fixed seed (251), so the split is byte-identical on every regeneration and is not committed (splits/val_split.csv is gitignored and regenerated once per checkout). Every ablation and architecture comparison in this report reads val-dev; val-test is read exactly once, for the headline number in Section 5.3.

87 **Proxy protocol.** At the measured throughput of the selected trunk (about 1.2 minutes/epoch,
88 Section 5.1), a 15-epoch run costs about 18 minutes, making comparisons that would be unaffordable
89 at full length routine. Every architecture and recipe comparison below ranks candidates on a fixed-
90 seed, 15-epoch val-dev proxy; the winning configuration is then confirmed with one full-length run.

91 **Logging.** Every run cited in this report is logged with a config hash, per-epoch train/validation
92 top-1 and top-5, wall-clock time, and peak VRAM (`src/runlog.py`), so every number below traces
93 back to a specific run.

94 4 Method

95 This section describes FoodCNN’s architecture, then the training recipe used to fit it.

96 4.1 Architecture

97 FoodCNN (`src/model.py`) is a plain residual CNN, sized to spend its parameter budget where it
98 can affect accuracy rather than where convention places it. A stem (3x3 stride-2 convolution, batch
99 normalization, ReLU, 3x3 stride-2 max-pool) takes the input to a quarter of its input resolution before
100 any residual stage runs. Four residual stages follow, each halving spatial resolution and doubling
101 channel width relative to the last – 64, 128, 256, then 512 channels, with block counts [2, 2, 2, 1] –
102 so that the final stage, carrying one block rather than two, is what keeps the trunk under budget.

103 Each residual block places batch normalization after both convolutions in the main branch and,
104 whenever the shortcut is not the identity, after the 1x1 shortcut projection as well – an unnormalized
105 branch should not be added to a normalized one. The second batch norm’s scale (γ) in every block is
106 zero-initialized, so every block begins training as an identity map; this is what makes a high learning
107 rate safe under warmup. The nonlinearity is applied after the residual addition, not before it, so each
108 block is an actual nonlinearity rather than an affine detour.

109 **A genuinely global head.** Pooling to (1, 1) rather than the (7, 7) map convolutions naturally leave
110 behind matters: at (7, 7), flattening produces a $512 \times 7 \times 7 = 25,088$ -feature vector, and a single
111 subsequent linear layer to 251 classes costs 6.4M parameters – 94% of the entire model, spent on
112 its least expressive layer. Pooling to (1, 1) first reduces that same layer to `Linear(512, 251)`, 129k
113 parameters.

114 **Parameter budget.** With the fixes above, FoodCNN totals 6,576,955 trainable parameters – 65.8%
115 of the 10M budget – with roughly 98% of them inside the convolutional trunk rather than the
116 classifier head.

117 **A rejected alternative.** A depthwise-separable version of the same trunk (MobileNetV2-style
118 inverted residuals, retained in `src/model.py` as a documented negative result rather than deleted) was
119 measured head-to-head with the plain-convolution trunk on this hardware: 2.55M parameters, 621
120 img/s, 3.83 GiB peak memory, against the plain-convolution trunk’s 6.58M parameters, 918 img/s,
121 1.28 GiB peak (full detail in Section C). Depthwise convolutions here cost activation memory and
122 wall-clock far more than they save in parameter count, and under this project’s constraints, throughput
123 and memory – not parameter count – are what actually bind (Section 6).

124 4.2 Training recipe

125 The reference training loop is modernized in ways that cost nothing at this parameter budget:
126 `torch.autocast("cuda", dtype=torch.bfloat16)` without a `GradScaler` (bf16 does not need
127 dynamic loss scaling), `channels_last` memory format on both model and inputs, a cosine learning-
128 rate schedule with five-epoch linear warmup in place of step decay, `CrossEntropyLoss` with label
129 smoothing 0.1, SGD with Nesterov momentum, eight dataloader workers (up from four), a 176px
130 training crop (down from 224), and no `DataParallel` wrapper – there is only one GPU to shard
131 across, and the wrapper’s scatter/gather adds a per-step cost and a `module.` prefix to every checkpoint
132 key for no benefit here.

133 4.3 Recipe extensions

Table 1: 15-epoch val-dev proxy results for four candidate trunks, all under the 10M-parameter cap.

Variant	Params	val-dev top-1	val-dev top-5	Peak VRAM
[2,2,2,2] 64-448	9.46M	49.99%	78.29%	2.04 GiB
[2,2,4,1] 64-512	8.94M	49.73%	77.93%	2.14 GiB
baseline [2,2,2,1] 64-512	6.58M	48.82%	77.12%	1.99 GiB
5-stage 48-512 [2,2,2,2,1]	9.08M	47.42%	75.94%	1.61 GiB

Four further axes are implemented as opt-in flags to the training script, all defaulting to off so that nothing about the recipe above changes unless explicitly requested: an augmentation switch between no augmentation, TrivialAugment [6], and RandAugment [5]; batch-wise Mixup [7] and CutMix [8], scored against the pre-mix hard label since a mixed batch has no single correct class to evaluate top-1/top-5 against; an exponential moving average of model weights (`torch.optim.swa_utils.AveragedModel`, batch-norm running statistics left live rather than averaged); and Generalized Cross Entropy [9] as an alternative to label-smoothed cross-entropy for the training set’s label noise. Section 5.2 reports which, if any, improve on the recipe above.

5 Experiments

Accuracy figures below are val-dev top-1/top-5 unless stated otherwise; val-test is read exactly once, in Section 5.3, per the protocol in Section 3. All measurements were taken on the project GPU (RTX 5050 Laptop, 8GB VRAM) in its performance power profile; Section 6 explains why that qualifier matters for every throughput number below.

5.1 Architecture search

A first pass screened trunk variants by synthetic throughput and memory alone (benchmarks/bench.py, benchmarks/trunk_variants.py, full sweep in Section B), before spending any GPU-hours on accuracy. Two findings from that pass decided which variants were worth an accuracy proxy at all. First, stage widths must be multiples of 32: a 72-144-288-576 trunk has **fewer** parameters than the five-stage candidate below yet runs 46% slower, falling off the tensor-core fast path. Second, final spatial resolution dominates cost more than any other single knob: dropping the stem’s max-pool to preserve a 6x6 final feature map, at identical parameter count, costs 2.9x throughput (1960 to 684 img/s) and doubles peak memory.

Four candidates under the parameter cap were then measured for accuracy with fixed-seed, 15-epoch val-dev proxies (benchmarks/proxy_sweep.py):

The five-stage variant’s extra downsample to a 3x3 final map costs 1.4-2.6 accuracy points despite being the second-largest candidate by parameter count, so the throughput and memory advantage it showed in the synthetic pass does not transfer to a real training signal; it is ruled out. The remaining three sit within 1.2 points of each other on a single seed each – not enough to call a winner with confidence. We adopt the baseline [2,2,2,1] 64-512 trunk provisionally: it is statistically indistinguishable from the other two here, uses 30% fewer parameters than the nominal leader, and that headroom has not measured as worth anything yet – better spent, for now, on the recipe tuning in Section 5.2. [2,2,4,1] and [2,2,2,2] 64-448 remain defined in benchmarks/trunk_variants.VARIANTS and can be re-proxied without repeating the measurement work above, if the tuned baseline underperforms expectations.

5.2 Recipe ablation

Six training-recipe axes were swept as fixed-seed, 15-epoch val-dev proxies on the selected trunk (benchmarks/proxy_sweep.py), confirmed with paired McNemar’s tests on the shared val-dev predictions wherever the point-estimate gap was close enough to need one (benchmarks/significance_test.py).

Table 2: Recipe axes ablated against the plain, label-smoothed cross-entropy control (15-epoch val-dev proxy unless noted), on the selected trunk. Deltas and significance are paired McNemar’s tests on the same val-dev images.

Axis	val-dev top-1	Δ vs. control	McNemar p
Control (plain, lr=0.8)	55.58%	–	–
TrivialAugment	52.09%	–3.49	not tested
RandAugment (15ep)	52.60%	–2.98	not tested
RandAugment (30ep, matched)	60.43% ¹	–0.84	0.0750
Mixup	42.95%	–12.63	< 0.0001
CutMix	48.84%	–6.74	< 0.0001
EMA (decay 0.999)	39.88%	–15.70	< 0.0001
GCE loss ($q = 0.7$)	22.23%	–33.35	< 0.0001
Batch 160 (lr=0.5)	55.85%	+0.18	0.7407
Batch 512 (lr=1.6)	55.30%	–0.36	0.4868
Crop-scale 0.25	55.71%	+0.05	0.9465
Crop-scale 0.40	54.49%	–1.17	0.0232

Learning rate. Sweeping $\{0.05, 0.1, 0.2, 0.4\}$ at batch 256 gave a clean diminishing-returns curve (43.96%, 49.41%, 52.56%, 54.02% top-1); extending to $\{0.6, 0.8\}$ continued the pattern (54.96%, 55.58%), each doubling buying roughly half the previous gain (+5.45, +3.15, +1.46, +0.94, +0.62 points). We stop at **lr=0.8**: the next doubling would buy an estimated 0.3-0.4 points against a visibly noisier validation curve.

Augmentation. TrivialAugment and RandAugment both trail the no-augmentation control by about 3 points at 15 epochs (52.09%, 52.60% vs. 55.58%), but their train loss is still well above the control’s at that point, so a matched 30-epoch rematch was run for RandAugment: the gap narrows to 0.84 points (61.27% vs. 60.43%), and a paired McNemar’s test on the discordant val-dev predictions finds it **not statistically significant** ($p = 0.075$, $n = 855$ discordant). Plain carries forward on an unbeaten record, not a proven margin over RandAugment specifically.

Mixup, CutMix, EMA, and Generalized Cross Entropy. All four lose decisively against the 15-epoch control (55.58%): Mixup 42.95%, CutMix 48.84%, EMA (decay 0.999) 39.88%, and GCE ($q = 0.7$) 22.23%, each at $p < 0.0001$. GCE’s collapse is the most striking – 119 of 251 classes reach zero accuracy, confidently wrong rather than merely underconfident (mean confidence on wrong predictions 0.49, the worst calibration of any recipe tested, ECE 0.328) – consistent with lr=0.8 being tuned for cross-entropy’s loss landscape rather than GCE’s bounded one. EMA’s collapse is consistent with its roughly 1,000-step averaging window not having caught up within this proxy’s roughly 6,945 steps. Neither failure rules out the technique outright; both would need their own learning-rate search to judge fairly, which this budget does not spend.

Batch size. Batch 160/256/512, with learning rate scaled linearly (0.5/0.8/1.6), land within 0.36 points of each other and every pairwise McNemar’s test is non-significant ($p \geq 0.49$). Batch size is a non-lever for this recipe at fixed resolution, matching the flat throughput already measured in Section 5.1.

Crop-scale floor. Raising RandomResizedCrop’s minimum scale from the ImageNet-inherited 0.08 to 0.25 or 0.40 closes the train/validation gap that motivated the sweep (+11.05 to +4.31 to –0.56 points), but validation accuracy does not follow: 0.25 ties the control and 0.40 is significantly worse (–1.17 points, $p = 0.0232$). The low-scale crop is doing double duty as both regularizer and view-diversity source; relaxing it trades away diversity without buying back epochs. Default stays at 0.08.

¹Against a 30-epoch control at 61.27%; not comparable to the 15-epoch control row above.

Adopted recipe. Nothing tested beats the plain recipe – label-smoothed cross-entropy, no extra augmentation, no Mixup/CutMix, no EMA, at **lr=0.8, batch 256, crop-scale-min 0.08** – and it carries forward unmodified into the full run (Section 5.3). RandAugment is the one axis not decisively ruled out: its matched-epoch gap is real in direction but not in significance, and it remains the first candidate worth revisiting if the full run underperforms.

5.3 Full supervised training run

A full 90-epoch run on the unmodified baseline trunk, with the training recipe above and none of Section 5.2’s extensions, validated that the pipeline survives full length before spending further GPU-hours tuning it. It converged to **61.57% val-dev top-1 / 85.67% top-5** (best checkpoint at epoch 82: 62.08% / 85.52%), rising from about 7% at epoch 0 and comfortably past both a from-scratch Food-101 anchor (mid-50s%) and this trunk’s own 15-epoch proxy (Table 1, 47-50%). No crashes, NaNs, or out-of-memory errors occurred across the full run; peak memory was 1.99 GiB, still far under the 8 GiB ceiling at full length. Wall-clock was 2.31 hours, 28% above the approximately 1.8 hour synthetic estimate from the architecture search’s throughput sweep – per-batch time rose from about 0.16s early in the run to about 0.26s past epoch 60 as sustained load heat-soaked the laptop GPU (Section 6). This run validates the pipeline; it is not the project’s final supervised number, which is what Section 5.2’s tuned recipe produces, read once against val -test with FixRes-style test-time resolution correction [14] (train at 176px, evaluate at 224px center crop).

The tuned recipe adopted in Section 5.2 – the same trunk and GAP head, plain label-smoothed cross-entropy, lr=0.8, batch 256, crop-scale-min 0.08, every setting the recipe search settled on, unmodified – was then trained for the same 90 epochs. It reached **64.36% val-dev top-1 / 87.51% top-5** at its best checkpoint (epoch 86), and, read against val -test exactly once, **63.83% top-1 / 81.79% top-3 / 87.39% top-5** – this report’s headline number. The run took 2h05m wall-clock at 1.99 GiB peak VRAM, and the train/validation gap that motivated the crop-scale investigation above closed from +11.05 points at epoch 15 to near zero by epoch 90 (final-epoch train top-1 64.88% vs. val top-1 64.26%): the model needed the full schedule to converge, and nothing in Section 5.2 sped that up because, per the crop-scale result, nothing needed to.

5.4 Self-supervised comparison

TODO — Not started. Planned: SimSiam/BYOL pretraining for 200 epochs at 176px (about 8 GPU-hours), finetuned at 176px (about 1.8 GPU-hours), compared against an equal-GPU-hour supervised control (about 8 GPU-hours) trained on the same trunk – without that control, the comparison cannot separate “self-supervised pretraining helped” from “this trunk keeps improving with more epochs regardless of objective.” SimCLR [10] is excluded: it is known to degrade below roughly 1,000-example batches, unreachable within an 8GB budget. Results replace this paragraph once measured.

6 Discussion

Hardware measurement mattered as much as the architecture. Several findings above only exist because throughput and memory were measured rather than assumed. Stage widths must be multiples of 32 to stay on the tensor-core fast path (a 46% throughput gap at equal parameter count, Section 5.1); final spatial resolution costs more than almost any other knob (2.9x throughput, 2x memory, for an unchanged parameter count); the dataloader is never the bottleneck at this resolution – plain crop-and-flip sustains 4.4-4.7k images/second at any worker count against roughly 1.7-1.9k images/second of GPU demand, and even TrivialAugment keeps 2.5x headroom at eight workers, falling to 1.4x at sixteen as oversubscription costs throughput on this machine’s 15GB of system RAM; and batch size is throughput-flat from 160 to 768 (1738-1785 img/s) while memory scales linearly from 1.28 to 5.64 GiB, meaning the GPU is compute-saturated at the smallest batch size tried, not idle.

Thermal throttling makes a short benchmark an upper bound, not an estimate. The GPU runs against a roughly 50W power cap with its clock oscillating 2175-2340 MHz against a 3090 MHz maximum, and an early version of the throughput benchmark swung 45% on one variant between sweeps before being changed to time three 40-step rounds and keep the best (sweeps now agree within

275 2-5%, and rankings are stable). Even so, the full run in Section 5.3 took 28% longer than its own
276 synthetic estimate: sustained 90-epoch load heat-soaks this laptop GPU in a way no short benchmark
277 window reproduces. Every throughput figure in this report is an upper bound on real epoch time and
278 a ratio between variants, not a wall-clock promise.

279 **Limitations.** The architecture comparison in Section 5.1 rests on a single 15-epoch seed per
280 variant; the 1.2-point spread separating three of the four candidates is well within plausible run-to-
281 run noise, which is why the baseline trunk is adopted provisionally rather than declared a winner
282 outright. All measurements are specific to one machine (RTX 5050 Laptop, 8GB VRAM) and need
283 not transfer, even in ratio, to other hardware. Training is from scratch throughout, by the project’s
284 own constraint, so absolute accuracy figures here are not directly comparable to pretrained food-
285 classification baselines without accounting for that.

286 7 Conclusion

287 Under a 10M-parameter budget and a single 8GB laptop GPU, a plain residual CNN with a genuinely
288 global pooling head reaches 61.57% val-dev top-1 / 85.67% top-5 on FoodX-251 in a full 90-epoch
289 run, before any recipe tuning. Reaching that number meant treating measured wall-clock and memory
290 behavior – channel-width alignment, spatial resolution, thermal throttling – as design constraints
291 alongside the parameter cap itself: a fifth residual stage and a depthwise-separable trunk both looked
292 attractive on paper and measurably lost once actually run. The recipe ablation (Section 5.2) and the
293 self-supervised comparison against an equal-compute supervised control (Section 5.4) remain open.

294 **TODO — Revisit this conclusion once Section 5.2 and Section 5.4 have real results.**

295 References

- 296 [1] P. Kaur, K. Sikka, W. Wang, S. Belongie, and A. Divakaran, “FoodX-251: A Dataset for Fine-grained
297 Food Classification,” *arXiv preprint arXiv:1907.06167*, 2019, [Online]. Available: [https://arxiv.org/abs/](https://arxiv.org/abs/1907.06167)
298 1907.06167
- 299 [2] D. Qin *et al.*, “MobileNetV4 - Universal Models for the Mobile Ecosystem,” *arXiv preprint*
300 *arXiv:2404.10518*, 2024, [Online]. Available: <https://arxiv.org/abs/2404.10518>
- 301 [3] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, “MobileOne: An Improved One millisecond
302 Mobile Backbone,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recog-*
303 *nition*, 2022, pp. 7907–7917. [Online]. Available: <https://arxiv.org/abs/2206.04040>
- 304 [4] S. Woo *et al.*, “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” in
305 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16133–
306 16142. [Online]. Available: <https://arxiv.org/abs/2301.00808>
- 307 [5] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical automated data augmentation
308 with a reduced search space,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and*
309 *Pattern Recognition Workshops*, 2020, pp. 702–703. [Online]. Available: [https://arxiv.org/abs/1909.](https://arxiv.org/abs/1909.13719)
310 13719
- 311 [6] S. G. Müller and F. Hutter, “TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation,” in
312 *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021. [Online]. Available:
313 <https://arxiv.org/abs/2103.10158>
- 314 [7] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,”
315 in *International Conference on Learning Representations*, 2018. [Online]. Available: [https://arxiv.org/](https://arxiv.org/abs/1710.09412)
316 [abs/1710.09412](https://arxiv.org/abs/1710.09412)
- 317 [8] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “CutMix: Regularization Strategy to Train Strong
318 Classifiers with Localizable Features,” in *Proceedings of the IEEE/CVF International Conference on*
319 *Computer Vision*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.04899>
- 320 [9] Z. Zhang and M. Sabuncu, “Generalized Cross Entropy Loss for Training Deep Neural Networks with
321 Noisy Labels,” in *Advances in Neural Information Processing Systems*, 2018. [Online]. Available: [https://](https://arxiv.org/abs/1805.07836)
322 arxiv.org/abs/1805.07836

Table 3: Synthetic throughput/memory sweep over trunk variants under the 10M-parameter cap, 176px training resolution, batch 160. “90 ep” is the projected wall-clock for a full 90-epoch run at the measured throughput; “Final map” is the trunk’s spatial resolution immediately before global average pooling.

Variant	Params	img/s	90 ep	Peak VRAM	Final map
baseline [2, 2, 2, 1] 64-512	6.58M	1688	1.8 h	1.26 GiB	6x6
[2, 2, 3, 1] 64-512	7.76M	1585	1.9 h	1.31 GiB	6x6
[2, 2, 4, 1] 64-512	8.94M	1475	2.0 h	1.36 GiB	6x6
[2, 3, 3, 1] 64-512	8.06M	1460	2.0 h	1.39 GiB	6x6
[2, 2, 2, 2] 64-448	9.46M	1608	1.8 h	1.30 GiB	6x6
wider@11 64-320-512	7.98M	1494	2.0 h	1.30 GiB	6x6
5 stages 48-512 [2, 2, 2, 2, 1]	9.08M	1960	1.5 h	1.04 GiB	3x3
uniformly wider 72-576	8.31M	1062	2.8 h	1.41 GiB	6x6
5 stages, no stem max-pool	9.08M	684	4.3 h	2.16 GiB	6x6

- [10] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 1597–1607. [Online]. Available: <https://arxiv.org/abs/2002.05709>
- [11] J.-B. Grill *et al.*, “Bootstrap Your Own Latent - A New Approach to Self-Supervised Learning,” in *Advances in Neural Information Processing Systems*, 2020, pp. 21271–21284. [Online]. Available: <https://arxiv.org/abs/2006.07733>
- [12] X. Chen and K. He, “Exploring Simple Siamese Representation Learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. [Online]. Available: <https://arxiv.org/abs/2011.10566>
- [13] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, “Barlow Twins: Self-Supervised Learning via Redundancy Reduction,” in *Proceedings of the 38th International Conference on Machine Learning*, 2021, pp. 12310–12320. [Online]. Available: <https://arxiv.org/abs/2103.03230>
- [14] H. Touvron, A. Vedaldi, M. Douze, and H. Jégou, “Fixing the train-test resolution discrepancy,” *Advances in Neural Information Processing Systems*, vol. 32, 2019, [Online]. Available: <https://arxiv.org/abs/1906.06423>

A Technical appendices and supplementary material

Supplementary tables referenced from Section 4 and Section 5.1 follow: the full synthetic throughput sweep behind the architecture search, and additional detail on the rejected depthwise-separable trunk.

B Full architecture-screening sweep

Table 1 in the main text reports 15-epoch accuracy proxies for four trunk variants. Those four were selected from a larger synthetic screening pass over throughput, parameter count, and memory alone (benchmarks/bench.py, benchmarks/trunk_variants.py), all at 176px training resolution and batch 160, before any accuracy was measured. The full sweep:

The last row isolates the resolution effect cited in Section 6: identical parameter count to the row above it, but with the stem’s max-pool removed to keep the final feature map at 6x6 instead of 3x3, at a cost of 2.9x throughput and roughly 2x peak memory. The “uniformly wider 72-576” row isolates the tensor-core alignment effect: fewer parameters than the five-stage variant, at 46% lower throughput, purely from using channel widths that are not multiples of 32.

C The rejected depthwise-separable trunk

366 Table 4: Depthwise-separable vs. plain-convolution trunk, measured head-to-head on the project
 367 GPU (176px, batch 160, bf16, channels_last).

368	Trunk	Params	img/s	Peak VRAM
369	Depthwise-separable (MobileNetV2-style)	2.55M	621	3.83 GiB
370	Plain-convolution residual (adopted)	6.58M	918	1.28 GiB

371 Before the residual trunk in Section 4, a MobileNetV2-style depthwise-separable version of the
 372 same network was implemented and measured head-to-head on the same hardware. It is kept in `src/
 373 model.py`, unused by `FoodCNN.features`, as a documented negative result rather than deleted:

374 Fewer parameters did not mean faster or lighter here: depthwise convolutions trade parameter count
 375 for activation memory and wall-clock, and under this project’s binding constraints – throughput and
 376 memory, not raw parameter count – that trade loses.² The plain-convolution trunk was adopted on
 377 this basis before the architecture search in Section 5.1 ever began.

²This comparison’s 918 img/s baseline figure is lower than the 1688 img/s recorded for the same trunk in Table 3. The two were measured in separate benchmarking passes – this comparison predates `benchmarks/bench.py`’s current three-round, keep-the-best methodology – and, per the thermal-throttling behavior discussed in Section 6, absolute throughput on this hardware is not stable across passes. The qualitative conclusion, that the depthwise-separable trunk trades parameters for memory and wall-clock, does not depend on which figure is used.

378 NeurIPS Paper Checklist

379 The checklist is designed to encourage best practices for responsible machine learning research,
380 addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove
381 the checklist: **The papers not including the checklist will be desk rejected.** The checklist should
382 follow the references and follow the (optional) supplemental material. The checklist does NOT count
383 towards the page limit.

384 Please read the checklist guidelines carefully for information on how to answer these questions. For
385 each question in the checklist:

- 386 • You should answer [Yes], [No], or [NA].
- 387 • [NA] means either that the question is Not Applicable for that particular paper or the
388 relevant information is Not Available.
- 389 • Please provide a short (1–2 sentence) justification right after your answer (even for NA).

390 **The checklist answers are an integral part of your paper submission.** They are visible to the
391 reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it
392 (after eventual revisions) with the final version of your paper, and its final version will be published
393 with the paper.

394 The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation.
395 While “[Yes]” is generally preferable to “[No]”, it is perfectly acceptable to answer “[No]” provided a
396 proper justification is given (e.g., “error bars are not reported because it would be too computationally
397 expensive” or “we were unable to find the license for the dataset we used”). In general, answering
398 “[No]” or “[NA]” is not grounds for rejection. While the questions are phrased in a binary way, we
399 acknowledge that the true answer is often more nuanced, so please just use your best judgment and
400 write a justification to elaborate. All supporting evidence can appear either in the main paper or the
401 supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification
402 please point to the section(s) where related material for the question can be found.

403 IMPORTANT, please:

- 404 • **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- 405 • **Keep the checklist subsection headings, questions/answers and guidelines below.**
- 406 • **Do not modify the questions and only use the provided macros for your answers.**

407 1. Claims

408 Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s
409 contributions and scope?

410 Answer: **[TODO]**

411 Justification: **[TODO]**

412 Guidelines:

- 413 • The answer NA means that the abstract and introduction do not include the claims made in
414 the paper.
- 415 • The abstract and/or introduction should clearly state the claims made, including the contri-
416 butions made in the paper and important assumptions and limitations. A No or NA answer
417 to this question will not be perceived well by the reviewers.
- 418 • The claims made should match theoretical and experimental results, and reflect how much
419 the results can be expected to generalize to other settings.
- 420 • It is fine to include aspirational goals as motivation as long as it is clear that these goals are
421 not attained by the paper.

422 2. Limitations

423 Question: Does the paper discuss the limitations of the work performed by the authors?

424 Answer: **[TODO]**

425 Justification: **[TODO]**

426 Guidelines:

- 427 • The answer NA means that the paper has no limitation while the answer No means that the
- 428 paper has limitations, but those are not discussed in the paper.
- 429 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 430 • The paper should point out any strong assumptions and how robust the results are to
- 431 violations of these assumptions (e.g., independence assumptions, noiseless settings, model
- 432 well-specification, asymptotic approximations only holding locally). The authors should
- 433 reflect on how these assumptions might be violated in practice and what the implications
- 434 would be.
- 435 • The authors should reflect on the scope of the claims made, e.g., if the approach was only
- 436 tested on a few datasets or with a few runs. In general, empirical results often depend on
- 437 implicit assumptions, which should be articulated.
- 438 • The authors should reflect on the factors that influence the performance of the approach. For
- 439 example, a facial recognition algorithm may perform poorly when image resolution is low
- 440 or images are taken in low lighting. Or a speech-to-text system might not be used reliably
- 441 to provide closed captions for online lectures because it fails to handle technical jargon.
- 442 • The authors should discuss the computational efficiency of the proposed algorithms and
- 443 how they scale with dataset size.
- 444 • If applicable, the authors should discuss possible limitations of their approach to address
- 445 problems of privacy and fairness.
- 446 • While the authors might fear that complete honesty about limitations might be used by
- 447 reviewers as grounds for rejection, a worse outcome might be that reviewers discover limita-
- 448 tions that aren’t acknowledged in the paper. The authors should use their best judgment and
- 449 recognize that individual actions in favor of transparency play an important role in devel-
- 450 oping norms that preserve the integrity of the community. Reviewers will be specifically
- 451 instructed to not penalize honesty concerning limitations.

452 3. Theory Assumptions and Proofs

453 Question: For each theoretical result, does the paper provide the full set of assumptions and a

454 complete (and correct) proof?

455 Answer: **[TODO]**

456 Justification: **[TODO]**

457 Guidelines:

- 458 • The answer NA means that the paper does not include theoretical results.
- 459 • All the theorems, formulas, and proofs in the paper should be numbered and cross-
- 460 referenced.
- 461 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 462 • The proofs can either appear in the main paper or the supplemental material, but if they
- 463 appear in the supplemental material, the authors are encouraged to provide a short proof
- 464 sketch to provide intuition.
- 465 • Inversely, any informal proof provided in the core of the paper should be complemented by
- 466 formal proofs provided in appendix or supplemental material.
- 467 • Theorems and Lemmas that the proof relies upon should be properly referenced.

468 4. Experimental Result Reproducibility

469 Question: Does the paper fully disclose all the information needed to reproduce the main exper-
470 imental results of the paper to the extent that it affects the main claims and/or conclusions of the
471 paper (regardless of whether the code and data are provided or not)?

472 Answer: **[TODO]**

473 Justification: **[TODO]**

474 Guidelines:

- 475 • The answer NA means that the paper does not include experiments.
- 476 • If the paper includes experiments, a No answer to this question will not be perceived well by
477 the reviewers: Making the paper reproducible is important, regardless of whether the code
478 and data are provided or not.
- 479 • If the contribution is a dataset and/or model, the authors should describe the steps taken to
480 make their results reproducible or verifiable.
- 481 • Depending on the contribution, reproducibility can be accomplished in various ways. For
482 example, if the contribution is a novel architecture, describing the architecture fully might
483 suffice, or if the contribution is a specific model and empirical evaluation, it may be
484 necessary to either make it possible for others to replicate the model with the same dataset,
485 or provide access to the model. In general, releasing code and data is often one good way
486 to accomplish this, but reproducibility can also be provided via detailed instructions for
487 how to replicate the results, access to a hosted model (e.g., in the case of a large language
488 model), releasing of a model checkpoint, or other means that are appropriate to the research
489 performed.
- 490 • While NeurIPS does not require releasing code, the conference does require all submissions
491 to provide some reasonable avenue for reproducibility, which may depend on the nature of
492 the contribution. For example
 - 493 (a) If the contribution is primarily a new algorithm, the paper should make it clear how to
494 reproduce that algorithm.
 - 495 (b) If the contribution is primarily a new model architecture, the paper should describe the
496 architecture clearly and fully.
 - 497 (c) If the contribution is a new model (e.g., a large language model), then there should either
498 be a way to access this model for reproducing the results or a way to reproduce the model
499 (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - 500 (d) We recognize that reproducibility may be tricky in some cases, in which case authors
501 are welcome to describe the particular way they provide for reproducibility. In the case
502 of closed-source models, it may be that access to the model is limited in some way (e.g.,
503 to registered users), but it should be possible for other researchers to have some path to
504 reproducing or verifying the results.

505 5. Open Access to Data and Code

506 Question: Does the paper provide open access to the data and code, with sufficient instructions
507 to faithfully reproduce the main experimental results, as described in supplemental material?

508 Answer: **[TODO]**

509 Justification: **[TODO]**

510 Guidelines:

- 511 • The answer NA means that paper does not include experiments requiring code.
- 512 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
513 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 514 • While we encourage the release of code and data, we understand that this might not be
515 possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including
516 code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental Setting/Details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment Statistical Significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer “Yes” if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments Compute Resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader Impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point

607 out that an improvement in the quality of generative models could be used to generate
 608 deepfakes for disinformation. On the other hand, it is not needed to point out that a generic
 609 algorithm for optimizing neural networks could enable people to train models that generate
 610 Deepfakes faster.

- 611 • The authors should consider possible harms that could arise when the technology is being
 612 used as intended and functioning correctly, harms that could arise when the technology is
 613 being used as intended but gives incorrect results, and harms following from (intentional or
 614 unintentional) misuse of the technology.
- 615 • If there are negative societal impacts, the authors could also discuss possible mitigation
 616 strategies (e.g., gated release of models, providing defenses in addition to attacks, mecha-
 617 nisms for monitoring misuse, mechanisms to monitor how a system learns from feedback
 618 over time, improving the efficiency and accessibility of ML).

619 11. Safeguards

620 Question: Does the paper describe safeguards that have been put in place for responsible release
 621 of data or models that have a high risk for misuse (e.g., pre-trained language models, image
 622 generators, or scraped datasets)?

623 Answer: **[TODO]**

624 Justification: **[TODO]**

625 Guidelines:

- 626 • The answer NA means that the paper poses no such risks.
- 627 • Released models that have a high risk for misuse or dual-use should be released with
 628 necessary safeguards to allow for controlled use of the model, for example by requiring that
 629 users adhere to usage guidelines or restrictions to access the model or implementing safety
 630 filters.
- 631 • Datasets that have been scraped from the Internet could pose safety risks. The authors should
 632 describe how they avoided releasing unsafe images.
- 633 • We recognize that providing effective safeguards is challenging, and many papers do not
 634 require this, but we encourage authors to take this into account and make a best faith effort.

635 12. Licenses for Existing Assets

636 Question: Are the creators or original owners of assets (e.g., code, data, models), used in the
 637 paper, properly credited and are the license and terms of use explicitly mentioned and properly
 638 respected?

639 Answer: **[TODO]**

640 Justification: **[TODO]**

641 Guidelines:

- 642 • The answer NA means that the paper does not use existing assets.
- 643 • The authors should cite the original paper that produced the code package or dataset.
- 644 • The authors should state which version of the asset is used and, if possible, include a URL.
- 645 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 646 • For scraped data from a particular source (e.g., website), the copyright and terms of service
 647 of that source should be provided.
- 648 • If assets are released, the license, copyright information, and terms of use in the package
 649 should be provided. For popular datasets, <https://paperswithcode.com/datasets> has
 650 curated licenses for some datasets. Their licensing guide can help determine the license of
 651 a dataset.
- 652 • For existing datasets that are re-packaged, both the original license and the license of the
 653 derived asset (if it has changed) should be provided.

654 • If this information is not available online, the authors are encouraged to reach out to the
655 asset's creators.

656 13. New Assets

657 Question: Are new assets introduced in the paper well documented and is the documentation
658 provided alongside the assets?

659 Answer: **[TODO]**

660 Justification: **[TODO]**

661 Guidelines:

- 662 • The answer NA means that the paper does not release new assets.
- 663 • Researchers should communicate the details of the dataset/code/model as part of their
664 submissions via structured templates. This includes details about training, license, limita-
665 tions, etc.
- 666 • The paper should discuss whether and how consent was obtained from people whose asset
667 is used.
- 668 • At submission time, remember to anonymize your assets (if applicable). You can either
669 create an anonymized URL or include an anonymized zip file.

670 14. Crowdsourcing and Research with Human Subjects

671 Question: For crowdsourcing experiments and research with human subjects, does the paper
672 include the full text of instructions given to participants and screenshots, if applicable, as well
673 as details about compensation (if any)?

674 Answer: **[TODO]**

675 Justification: **[TODO]**

676 Guidelines:

- 677 • The answer NA means that the paper does not involve crowdsourcing nor research with
678 human subjects.
- 679 • Including this information in the supplemental material is fine, but if the main contribution
680 of the paper involves human subjects, then as much detail as possible should be included in
681 the main paper.
- 682 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or
683 other labor should be paid at least the minimum wage in the country of the data collector.

684 15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human 685 Subjects

686 Question: Does the paper describe potential risks incurred by study participants, whether such
687 risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals
688 (or an equivalent approval/review based on the requirements of your country or institution) were
689 obtained?

690 Answer: **[TODO]**

691 Justification: **[TODO]**

692 Guidelines:

- 693 • The answer NA means that the paper does not involve crowdsourcing nor research with
694 human subjects.
- 695 • Depending on the country in which research is conducted, IRB approval (or equivalent)
696 may be required for any human subjects research. If you obtained IRB approval, you should
697 clearly state this in the paper.

- 698 • We recognize that the procedures for this may vary significantly between institutions and
699 locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines
700 for their institution.
- 701 • For initial submissions, do not include any information that would break anonymity (if
702 applicable), such as the institution conducting the review.

703 **16. Declaration of LLM Usage**

704 Question: Does the paper describe the usage of LLMs if it is an important, original, or non-
705 standard component of the core methods in this research? Note that if the LLM is used only for
706 writing, editing, or formatting purposes and does *not* impact the core methodology, scientific
707 rigor, or originality of the research, declaration is not required.

708 Answer: **[TODO]**

709 Justification: **[TODO]**

710 Guidelines:

- 711 • The answer NA means that the core method development in this research does not involve
712 LLMs as any important, original, or non-standard components.
- 713 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not be
714 described.